MLDLS · CHAPTER 01 · WORKSHOP 1

Tools and Libraries for Machine Learning

Setting up the environment and first steps with NumPy, Pandas, Matplotlib and Seaborn through simple examples

Module	Machine Learning, Deep Learning and Security (UEM7 · 4 credits · coefficient 2)
Specialty / Semester	Computer Security (Sécurité Informatique) · Semester 07
Chapter	Chapter 01 — Machine Learning
Session type	Workshop (TP) 1 of 3 · hands-on, individual
Duration	1 session of 1 h 30
Tools	Python 3, Jupyter Notebook or Google Colab, NumPy, Pandas, Matplotlib, Seaborn
Instructor	.....
Academic year	2026 – 2027

1. Overview

Chapter 1 lists the tools needed to practise machine learning on your own data: a Python notebook and four libraries. This first workshop installs the environment and gives a short, guided tour of each library with small examples taken from the lecture (the house-price table of §3.3, the spam table of §4.2) and from a tiny login log. No model is trained yet; the goal is that every student can create arrays, handle a table, and draw a plot before Workshop 2.

1.1 Learning outcomes

1. Open a notebook (Jupyter or Colab), run cells, install and import the four libraries.
2. NumPy: create arrays, read their shape, select elements, compute statistics, apply a formula to a whole matrix at once.
3. Pandas: build or load a table (DataFrame), inspect it, filter rows, add a column, group and count, handle missing values.
4. Matplotlib and Seaborn: draw a line, a bar chart, a histogram, a scatter plot and a box plot, with a title and labelled axes.

1.2 Prerequisites

- Basic Python (variables, lists, loops, functions).
- Chapter 1 lecture, Sections 1–3 (workflow, vocabulary X / Y / Θ, the house-price example).
- A Google account for Colab, or Anaconda installed.

1.3 Timetable (1 h 30)

Time	Part	Content
0:00 – 0:10	Part 0	The notebook, installing and importing the libraries
0:10 – 0:35	Part 1 — NumPy	Arrays, shape, indexing, statistics, one formula applied to a matrix (the house example)
0:35 – 1:00	Part 2 — Pandas	DataFrame, inspection, filtering, new column, groupby, missing values (login log)
1:00 – 1:20	Part 3 — Matplotlib & Seaborn	The five basic plots on the login log
1:20 – 1:30	Wrap-up	Short exercise, homework and checkpoint questions

2. Part 0 — The notebook and the libraries (10 min)

Option	How to start	Note
Google Colab (recommended)	colab.research.google.com → *New notebook*	Nothing to install; needs internet; files disappear when the session ends (download them)
Jupyter (Anaconda)	Anaconda Navigator → *Jupyter Notebook*, or type `jupyter notebook` in a terminal	Works offline; files stay on the machine

A notebook is a sequence of **cells**. Write code in a cell and press **Shift + Enter** to run it. The result appears under the cell. Variables stay in memory from one cell to the next.

Cell 0.1 — import the four libraries (run this first in every notebook)

```
import numpy as np                # numerical arrays
import pandas as pd               # tables
import matplotlib.pyplot as plt   # plots
import seaborn as sns             # nicer statistical plots, built on Matplotlib

print(np.__version__, pd.__version__)
```

If a line fails with `ModuleNotFoundError`, install the library from a cell with `!pip install seaborn` (the `!` runs a system command), then run the imports again.

Library	What it is for	Main object
NumPy	Fast numerical computation on arrays and matrices (the X and Y of the lecture)	`ndarray`
Pandas	Tables with named columns: load a CSV, clean, filter, group	`DataFrame`, `Series`
Matplotlib	Draw any plot, control every detail	`plt.figure`, `plt.plot`
Seaborn	Statistical plots from a DataFrame in one line	`sns.histplot`, `sns.boxplot`…

3. Part 1 — NumPy (25 min)

3.1 Create an array and read its shape

Cell 1.1

```
a = np.array([3, 1, 4, 1, 5])          # a vector (1 dimension)
M = np.array([[1, 2, 3],
              [4, 5, 6]])                 # a matrix (2 dimensions): 2 rows, 3 columns
print(a.shape, M.shape)                   # (5,) (2, 3)
print(np.zeros(3), np.ones((2, 2)), np.arange(0, 10, 2))
```

`shape` gives the size of each dimension. In machine learning the feature matrix **X** always has the shape `(number of samples, number of features)`: one row per observation, one column per feature.

3.2 The house-price table of the lecture as a matrix

Cell 1.2 — indexing

```
#          crime  area  rooms  floors  houses
X = np.array([[0.01,  250,    7,     3,    560],     # house 1
              [0.80,   60,    1,     7,    115],     # house 2
              [0.30,  120,    4,     2,      8]])    # house 3
y = np.array([500000, 130000, 120000])               # prices (the target Y)

print(X.shape)        # (3, 5) : 3 samples, 5 features
print(X[0])           # first row  = all features of house 1
print(X[:, 1])        # second column = the area of every house
print(X[0, 1])        # one value : area of house 1 -> 250.0
```

Read `X[row, column]`. A colon `:` means "everything". `X[:, 1]` therefore selects column 1 for all rows.

3.3 Statistics and filters

Cell 1.3

```
print(X[:, 1].mean())          # mean area  -> 143.3
print(X.max(axis=0))           # maximum of each column (axis=0 = down the rows)
print(X.mean(axis=1))          # mean of each row
print(X[X[:, 1] > 100])        # keep the houses whose area is above 100 (a filter)
print((X[:, 1] > 100).sum())   # how many of them -> 2
```

> **axis**
> `axis=0` works column by column (one result per feature), `axis=1` row by row (one result per sample). When in doubt, print the shape of the result.

3.4 One formula for the whole matrix: Step 3 of the lecture

In §3.3 the lecture computes the prediction of house 1 by hand with the random weights Θ = (0.6, 0.87, 0.301, 0.93, 0.4). NumPy does it for the three houses in one line with the matrix product `@`.

Cell 1.4

```
theta = np.array([0.6, 0.87, 0.301, 0.93, 0.4])
y_hat = X @ theta                      # Step 3 : prediction for every house at once
print(y_hat.round(1))                  # [446.4 105.5 110.8]
error = y_hat - y
print(((error) ** 2).mean())           # Step 4 : the cost J (mean squared error) -> huge
```

The result 446.4 for house 1 is the value computed in the lecture; the true price is 500 000, hence the enormous cost. This is why training (gradient descent) is needed — Workshop 2 lets Scikit-learn do it.

3.5 Scaling the features (preparation for Workshop 2)

Cell 1.5

```
X_scaled = (X - X.mean(axis=0)) / X.std(axis=0)   # subtract the mean, divide by the std, per column
print(X_scaled.round(2))
print(X_scaled.mean(axis=0).round(6))             # every column now has mean 0 ...
print(X_scaled.std(axis=0))                       # ... and standard deviation 1
```

Area is in the hundreds and *crime* below 1. Scaling puts every feature on the same scale so that no feature dominates the others; it is one line thanks to broadcasting (a matrix minus a vector = the vector subtracted from every row).

> **Exercise 1 (5 min)**
>
> The spam table of §4.2 is `S = np.array([[10, 4], [2, 1], [7, 3], [1, 2]])` (links, words in subject), weights Θ = (0.3, 0.7), bias 0.5.
>
> 1. Compute `z = S @ np.array([0.3, 0.7]) + 0.5` and check that the first value is 6.3 as in the lecture.
> 2. Apply the sigmoid `1 / (1 + np.exp(-z))` to the whole vector. Which e-mails are predicted as spam (probability ≥ 0.5)?

4. Part 2 — Pandas (25 min)

4.1 A DataFrame from a dictionary

Cell 2.1

```
houses = pd.DataFrame({"crime":  [0.01, 0.80, 0.30],
                       "area":   [250, 60, 120],
                       "rooms":  [7, 1, 4],
                       "price":  [0.50, 0.13, 0.12]},     # in millions
                      index=["H1", "H2", "H3"])
houses                              # a table with named columns and rows
houses["area"]                      # one column -> a Series
houses["area"].mean()               # 143.3
houses[houses["area"] > 100]        # filter rows, same idea as NumPy
houses["price_per_m2"] = houses["price"] * 1e6 / houses["area"]   # new column
houses
```

Compared with NumPy, columns have names and can hold different types (numbers, text, dates). Under the hood each column is a NumPy array: `houses.to_numpy()` gives back the matrix X.

4.2 A small login log: generate and save a CSV

Real data arrives as files. The cell below creates a log of 300 login events with a few missing values and a few suspicious bursts of failed attempts, and saves it as `logins.csv`.

Cell 2.2

```
rng = np.random.default_rng(1)
n = 300
log = pd.DataFrame({"user":    rng.choice(["ali", "sara", "yacine", "nour", "karim"], n),
                    "hour":    rng.integers(0, 24, n),
                    "failed_attempts": rng.poisson(0.4, n),
                    "country": rng.choice(["DZ", "FR", "TN", "RU"], n, p=[.7, .15, .1, .05]),
                    "duration_min": rng.gamma(2, 8, n).round(1)})
log.loc[rng.choice(n, 12, replace=False), "failed_attempts"] = rng.integers(5, 12, 12)  # bursts
log.loc[rng.choice(n, 10, replace=False), "duration_min"] = np.nan                      # missing
log.to_csv("logins.csv", index=False)
```

4.3 Load and inspect

Cell 2.3

```
df = pd.read_csv("logins.csv")
df.head()                 # first 5 rows
df.shape                  # (300, 5)
df.info()                 # column types and number of non-missing values
df.describe()             # statistics of the numerical columns
df["country"].value_counts()      # DZ 200, FR 45, TN 32, RU 23
df.isna().sum()                   # missing values per column -> duration_min 10
```

Always run these five commands on a new dataset. `info()` already answers three questions: how many rows, which columns are text, where values are missing.

4.4 Select, filter, sort

Cell 2.4

```
df[["user", "failed_attempts"]]                         # some columns
df.loc[0:4, "user":"failed_attempts"]                    # rows 0-4, columns from user to failed_attempts
suspicious = df[df["failed_attempts"] >= 5]              # a boolean filter
suspicious[["user", "country", "failed_attempts"]]
df[(df["failed_attempts"] >= 5) & (df["country"] != "DZ")]   # two conditions : & (and), | (or)
df.sort_values("failed_attempts", ascending=False).head()
```

> **Two classic errors**
>
> Use `&` and `|` with parentheses around each condition, never `and` / `or`. Column names are case-sensitive: `df["User"]` fails if the column is `user` (`df.columns` lists the exact names).

4.5 Group, count, cross-tabulate

Cell 2.5

```
df.groupby("user")["failed_attempts"].mean().round(2)       # mean failures per user
df.groupby("country").size()                                 # number of logins per country
pd.crosstab(df["country"], df["failed_attempts"] >= 5)       # country x suspicious ?
```

4.6 Missing values and a new feature

Cell 2.6

```
median = df["duration_min"].median()
df["duration_min"] = df["duration_min"].fillna(median)      # replace the 10 missing values
df["night"] = (df["hour"] < 6).astype(int)                  # new feature : 1 if the login is at night
df.groupby("night")["failed_attempts"].mean().round(2)
df.to_csv("logins_clean.csv", index=False)                  # save the cleaned table
```

> **Exercise 2 (5 min)**
>
> 1. How many logins come from Russia (`RU`) with at least 5 failed attempts?
> 2. Which user has the highest *total* number of failed attempts? (`groupby` + `sum` + `sort_values`)
> 3. Add a column `long_session` equal to 1 when `duration_min` is above 30.

5. Part 3 — Matplotlib and Seaborn (20 min)

Numbers alone hide the shape of the data; a plot shows it in a second. Matplotlib draws, Seaborn adds ready-made statistical plots that take a DataFrame directly. Every plot needs a title and axis labels.

Cell 3.1 — the five basic plots

```
# 1. Line plot : the cost decreasing during training (illustrative values, as in the lecture)
cost = [93.6, 40.1, 18.3, 9.0, 4.8, 2.9, 2.0, 1.6, 1.4, 1.3]
plt.plot(cost, marker="o")
plt.title("Cost J during gradient descent"); plt.xlabel("iteration"); plt.ylabel("cost")
plt.show()


# 2. Bar chart : logins per country
df["country"].value_counts().plot.bar(title="Logins per country")
plt.ylabel("count"); plt.show()


# 3. Histogram : distribution of session duration
sns.histplot(df["duration_min"], bins=30)
plt.title("Session duration (min)"); plt.show()


# 4. Scatter plot : hour vs failed attempts, coloured by country
sns.scatterplot(data=df, x="hour", y="failed_attempts", hue="country")
plt.title("Failed attempts by hour"); plt.show()


# 5. Box plot : failed attempts per user
sns.boxplot(data=df, x="user", y="failed_attempts")
plt.title("Failed attempts per user"); plt.show()
```

Plot	Question it answers	Command
Line	How does a value evolve (over time, over iterations)?	`plt.plot(values)`
Bar	How many per category?	`series.plot.bar()` or `sns.countplot`
Histogram	How is a numerical variable distributed? Skewed? Outliers?	`sns.histplot(col, bins=30)`
Scatter	Is there a relation between two numerical variables?	`sns.scatterplot(data=df, x=…, y=…, hue=…)`
Box plot	How does a variable differ between groups? Where are the outliers?	`sns.boxplot(data=df, x=group, y=value)`

Cell 3.2 — several plots in one figure, saved to a file

```
fig, axes = plt.subplots(1, 2, figsize=(10, 3.5))            # two plots side by side
sns.histplot(df["hour"], bins=24, ax=axes[0]); axes[0].set_title("Logins per hour")
sns.boxplot(data=df, x="country", y="duration_min", ax=axes[1]); axes[1].set_title("Duration by country")
plt.tight_layout()
plt.savefig("logins_figure.png", dpi=150)                     # save for a report
plt.show()
```

> **Exercise 3 (5 min)**
> Draw the histogram of `failed_attempts`. Why is a histogram better than the mean (0.7) to describe this column? Then draw a bar chart of the mean `failed_attempts` per `night` value.

6. Wrap-up, homework and assessment (10 min)

> **Checkpoint questions (answer in the notebook)**
>
> Q1. `X.shape` is `(300, 4)`. How many samples, how many features? What does `X[:, 2]` return?
>
> Q2. Give the Pandas command that shows the number of missing values in each column.
>
> Q3. Which plot would you use to see whether logins from `RU` have more failed attempts than logins from `DZ`?
>
> Q4. Why do we scale features before training a model (one sentence)?

6.1 Homework (for Workshop 2)

- Finish the three exercises and the checkpoint questions in `TP1_<name>.ipynb`, run **Kernel → Restart & Run All**, and submit the notebook with `logins_clean.csv`.
- Bonus: on `logins_clean.csv`, build the matrix `X` with the columns `hour`, `failed_attempts`, `duration_min`, `night` using `df[cols].to_numpy()`, scale it as in Cell 1.5, and print its shape. This is exactly the input Workshop 2 will feed to Scikit-learn.

6.2 Assessment (/10, continuous evaluation)

Criterion	Points
Notebook runs from top to bottom without error	2
NumPy: Exercise 1 correct (z and sigmoid values match the lecture)	2
Pandas: Exercise 2 correct, cleaned CSV produced	3
Plots: Exercise 3 with titles and labels, one-line interpretation	2
Checkpoint questions	1